

به نام خدا

راهنمای مقدماتی پایتون با ذهنیت میکروپایتون

تهیه و تنظیم: مهندس یاسین سموعی

کارشناسی ارشد الکترونیک دیجیتال

دانشگاه آزاد اسلامی واحد اصفهان

آذر ماه ۱۴۰۴



شروع کار با پایتون

- پایتون چیست: یک زبان سطح بالا، خوانا و چندمنظوره؛ مناسب وب، هوش مصنوعی، داده، و سیستم‌های نهفته.
- چرا برای IoT مناسب است: سینتکس ساده، کتابخانه‌های زیاد، و نسخه سبک MicroPython برای کنترل سخت‌افزار.
- نصب و اجرا:

- **Windows/macOS/Linux:** نصب از `python.org`، اجرای اسکریپت با `python file.py`
- **اجرای تعاملی (REPL):** اجرا با `python` و تست خطبه‌خط.
- **مدیر بسته‌ها:** نصب کتابخانه با `pip install ...`

نکته: برای تمرین IoT بدون برد، می‌توانید با ساختارها و فایل‌ها و شبکه در پایتون استاندارد تمرین کنید و بعد همان الگوها را روی بردها پیاده کنید.

مبانی زبان: متغیر، انواع داده، ورودی/خروجی، کامنت

متغیرها و انواع داده پایه

- **عدد صحیح (int):** مناسب شمارنده‌ها، وضعیت‌ها، مقادیر GPIO فرضی

```
x = 10
y = -3
print(x + y) # 7
```

- **عدد اعشاری (float):** مناسب خواندن سنسورهای آنالوگ (پس از تبدیل)

```
voltage = 3.3
temp_c = 24.75
print(voltage * temp_c) # 81.675
```

- **رشته (str):** مناسب پیام‌ها، فرمت لاگ و پروتکل‌های ساده

```
name = "Yasin"
message = f"Hello, {name}"
print(message)
```

- **بولین (bool):** مناسب وضعیت‌ها و شرط‌ها

```
is_on = True
if is_on:
```

```
print("Device is ON")
```

```
result = None
if result is None:
    print("No data yet")
```

```
print("Hello, world!")
state = True
print("State:", state)
```

```
user = input("Enter your name: ")
print("Welcome,", user)
```

```
def read_sensor():
    """Read sensor value and return normalized float in range [0,1]."""
    return 0.42
```

- **None**: نشان‌دهنده نبود مقدار

ورودی و خروجی

- خروجی با: **print**

- ورودی با **input**:

کامنت

- کامنت تک خطی:

#این یک کامنت است

- داک‌استرینگ برای توضیح تابع/ماژول:

رشته‌ها و اسلایسینگ به صورت کامل

عملیات پایه رشته‌ها

- طول، الحاق، تکرار:

```
s = "sensor"
print(len(s))      # 6
print(s + "_data") # "sensor_data"
print(s * 3)       # "sensorsensorsensor"
```

- دسترسی به کاراکترها:

```
s = "ESP32"
print(s[0])    # 'E'
print(s[-1])   # '2'
```

اسلایسینگ رشته‌ها

- فرمت کلی `s[start:stop:step]`:

```
s = "ABCDEFGH"
print(s[0:3])    # "ABC" (از ۰ تا ۳، شامل نیست)
print(s[:4])     # "ABCD" (شروع پیش فرض ۰)
print(s[3:])     # "DEFG" (تا انتها)
print(s[-4:])    # "DEFG" (چهار تای آخر)
print(s[:-2])    # "ABCDE" (به جز دو تای آخر)
print(s[::2])    # "ACEG" (گام ۲)
print(s[::-1])   # "GFEDCBA" (برعکس)
print(s[1:6:2])  # "BDF"
```

متدهای مهم رشته‌ها برای IoT

- پاک‌سازی ورودی و پارس ساده:

```
payload = " temp=24.7;humidity=41 "
payload = payload.strip()
pairs = payload.split(";") # ['temp=24.7', 'humidity=41']
kv = dict(p.split("=") for p in pairs)
print(kv) # {'temp': '24.7', 'humidity': '41'}
```

- جایگزینی و بررسی پیشوند/پسوند:

```
topic = "home/livingroom/temp"
print(topic.startswith("home/")) # True
print(topic.endswith("temp"))   # True
print(topic.replace("livingroom", "kitchen"))
```

- جستجو:

```
cmd = "LED=ON"
print("ON" in cmd) # True
print(cmd.find("LED")) # 0
print(cmd.count("=")) # 1
```

- فرمت کردن رشته‌ها:

```
value = 3.14159
print(f"v={value:.2f}")      # v=3.14
print("id: {}, status: {}".format(17, "OK"))
```

لیست‌ها: کامل، با اسلایسینگ و متدها

ایجاد و دسترسی

```
numbers = [1, 2, 3]
numbers.append(4) # [1, 2, 3, 4]
print(numbers[0]) # 1
print(numbers[-1]) # 4
print(numbers[1:]) # [2, 3, 4]
```

اسلایسینگ لیست

```
data = [10, 20, 30, 40, 50, 60]
print(data[:3]) # [10, 20, 30]
print(data[3:]) # [40, 50, 60]
print(data[-3:]) # [40, 50, 60]
print(data[::2]) # [10, 30, 50]
print(data[::-1]) # [60, 50, 40, 30, 20, 10]
```

متدهای مهم لیست‌ها

```
values = [3, 1, 4]
values.append(2) # [3, 1, 4, 2]
values.append([8, 13]) # [3, 1, 4, 2, 2, [8, 13]] اضافه کردن لیست تو در تو
values.extend([8, 13]) # [3, 1, 4, 2, 8, 13] اضافه کردن ۸ و ۱۳ آخر لیست به صورت جداگانه
values.insert(1, 99) # [3, 99, 1, 4, 2, 8, 13] اضافه کردن عدد ۹۹ در ایندکس ۱
removed = values.pop() # removed = 13 حذف آخرین؛
values.remove(99) # حذف اولین ۹۹
values.reverse() # معکوس
values.sort() # مرتب‌سازی صعودی
values.sort(reverse=True) # نزولی
print(values.index(4)) # موقعیت اولین ۴
print(values.count(1)) # تعداد ۱
```

مثال‌های IoT با لیست‌ها

- فیلتر کردن مقادیر غیرمنطقی:

```
samples = [0.1, -5, 0.3, 10.2, 0.4]
valid = [x for x in samples if 0 <= x <= 5]
```

```
print(valid) # [0.1, 0.3, 0.4]
```

تاپل‌ها: (Tuple) غیر قابل تغییر، سریع برای بسته‌بندی

ساخت و استفاده

```
coord = (12.0, 45.7) # 12.0 45.7
x, y = coord
print(x, y)
```

```
single = (42,) # تاپل یک‌عنصری نیاز به کاما دارد
empty = () حالت خالی
```

کاربردها

- کلید امن برای دیکشنری و داده ثابت:

```
device_key = ("room", "lamp", 1)
registry = {device_key: "ON"}
print(registry[("room", "lamp", 1)]) # ON
```

- بازگشت چند مقدار از تابع:

```
def read_temp_humid():
    return (24.3, 41)
t, h = read_temp_humid()
```

- اسلایسینگ تاپل مشابه لیست:

```
t = (1, 2, 3, 4)
print(t[1:3]) # (2, 3)
```

مجموعه‌ها: (Set) یکتا و عملیات ریاضی

ساخت و عملیات

```
seen_ids = {1, 2, 3}
seen_ids.add(4)
seen_ids.update([3, 5]) # اضافه کردن ۳ و ۵ به مجموعه {1, 2, 3, 5} فقط یکتا
seen_ids.remove(2) # اگر عددی که وجود ندارد حذف کنیم خطا میگیریم <= discard امنیت بیشتری دارد
seen_ids.discard(99) # حذف عدد ۹۹ از مجموعه بدون خطا
print(3 in seen_ids) # بررسی وجود عدد ۳ در مجموعه True
```

```
a = {1, 2, 3}
b = {3, 4}
print(a | b)    # {1,2,3,4}
print(a & b)    # {3}
print(a - b)    # {1,2}
```

کاربرد IoT

- جلوگیری از پردازش تکراری دستگاه‌ها:

```
processed = set()
for device_id in [101, 102, 101, 103]:
    if device_id in processed:
        continue
    # پردازش
    processed.add(device_id)
```

➔ **processed = {101, 102, 103}**

دیکشنری‌ها (Dictionary): کلید ← مقدار

ساخت و دسترسی

```
person = {"name": "Yasin", "age": 25}
print(person["name"]) # Yasin
person["city"] = "Khorramshahr"
print(person.get("country", "IR")) # مقدار پیش‌فرض اگر کلید وجود نداشت
```

متمدهای مهم دیکشنری

```
device = {"id": 17, "status": "OK", "temp": 24.7}

print(device.keys())    # dict_keys(['id', 'status', 'temp'])
print(device.values())  # dict_values([17, 'OK', 24.7])
print(device.items())   # dict_items([('id', 17), ('status', 'OK'), ('temp', 24.7)])

device.update({"status": "WARN", "voltage": 3.3})
removed = device.pop("temp", None) # با پیش‌فرض ایمن
device.setdefault("location", "lab") # اضافه اگر نبود

# پیمایش امن
for k, v in device.items():
    print(k, v)
```

کنترل جریان: شرطها و حلقهها

شرطها

```
temp = 25
if temp > 30:
    print("Hot")
elif temp < 15:
    print("Cold")
else:
    print("Comfort")
```

- شرطهای ترکیبی:

```
voltage = 3.2
current = 0.1
if 3.0 <= voltage <= 3.6 and current < 0.2:
    print("Safe range")
```

حلقهها

- **for** با **range**:

```
for i in range(5):
    print(i)
```

- **while**:

```
count = 0
while count < 5:
    print(count)
    count += 1
```

- حلقه روی لیستها:

```
devices = ["ESP32", "RP2040", "STM32"]
for d in devices:
    print(d)
```

- **break / continue / else** روی حلقه:


```
for x in [1, 3, 5, 7]:
    if x % 2 == 0:
        print("Even found")
        break # اتمام حلقه
else:
    print("No even number")
```

مثال برای `continue` و کاربرد آن :

رد کردن اعداد منفی:

```
numbers = [5, -2, 8, -1, 10, 3]
positive_sum = 0

for num in numbers:
    if num < 0:
        continue # از اعداد منفی رد شو
    positive_sum += num

print(f"مجموع اعداد مثبت: {positive_sum}") # خروجی: ۲۶
```

توابع: تعریف، آرگومان‌ها، مقدار بازگشتی

تعریف ساده

```
def greet(name):
    return "Hello " + name

print(greet("Yasin"))
```

آرگومان‌های پیش فرض و کلیدی

```
def read_adc(channel=0, scale=1.0):
    raw = 512 # فرضی
    return (raw / 1023) * scale

print(read_adc()) # scale 1.0
print(read_adc(channel=3)) # کانال ۳
print(read_adc(scale=3.3))
```

`**kwargs` و `*args`:

```
def calculate_average(*args):

    """ Takes any number of numbers and returns their average"""

    if not args:

        return 0

    return sum(args) / len(args)

#Usage:

print(calculate_average(1, 2, 3))          # 2.0

print(calculate_average(10, 20, 30, 40)) # 25.0

print(calculate_average())                 # 0


def build_profile(**kwargs):

    """Creates a user profile with flexible attributes"""

    profile = {}

    for key, value in kwargs.items():

        profile[key] = value

    return profile

# Usage:

user1 = build_profile(

    name="Alice",

    age=30,

    occupation="Engineer",

    city="New York"

)

user2 = build_profile(

    username="bob123",
```

```
        email="bob@email.com",

        is_premium=True

    )

    print(user1)

    # {'name': 'Alice', 'age': 30, 'occupation': 'Engineer', 'city': 'New York'}

    print(user2)

    # {'username': 'bob123', 'email': 'bob@email.com', 'is_premium': True}
```

خطاها و مدیریت استثنا (Exceptions)

try/except/else/finally

```
try:
    value = float("24.7")
except ValueError:
    value = 0.0
else:
    print("Parsed OK:", value) # فقط اگر کد با خطا مواجه نشد اجرا میشه که در این مثال اجرا میشه
finally:
    print("Cleanup resources if needed") # همیشه اجرا میشه
```

نمایش خطا داخل کنسول:

```
def safe_div(a, b):
    if b == 0:
        raise ZeroDivisionError("b cannot be zero")
    return a / b
```

کار با فایل‌ها (برای لاگ و پیکربندی)

خواندن/نوشتن

```
# نوشتن لاگ
with open("log.txt", "a", encoding="utf-8") as f:
    f.write("temp=24.7;humidity=41\n")
```

```
# خواندن
with open("log.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
```

JSON پیکربندی

```
import json

config = {"wifi_ssid": "mywifi", "wifi_pass": "secret", "sampling": 5}
with open("config.json", "w", encoding="utf-8") as f:
    json.dump(config, f, ensure_ascii=False, indent=2)

with open("config.json", "r", encoding="utf-8") as f:
    loaded = json.load(f)
print(loaded["sampling"])
```

اندیس دهی، پیمایش، zip و enumerate

```
data = [0.1, 0.2, 0.3]
for idx, val in enumerate(data):
    print(idx, val)

keys = ["temp", "humid"]
vals = [24.7, 41]
for k, v in zip(keys, vals):
    print(k, v)
```

ماژول ها و پکیج ها

import و alias

```
import math as m
print(m.sqrt(9))
```

ساختار فایل ها

- **file.py**: شامل توابع/کلاس ها
- **main.py**: نقطه ورود و `if __name__ == "__main__":`

```
# utils.py
def clamp(x, lo, hi):
    return max(lo, min(x, hi))
```

```
# main.py
from utils import clamp
print(clamp(1.2, 0, 1))
```

کلاس‌ها و OOP مختصر و کاربردی

تعریف کلاس ساده

```
class Sensor:
    def __init__(self, name, unit="C"):
        self.name = name
        self.unit = unit
        self._last = None

    def read(self):
        self._last = 24.7
        return self._last

    def __str__(self):
        return f"Sensor({self.name}, unit={self.unit})"

s = Sensor("temp")
print(s.read())
print(s)
```

ارث‌بری

```
class AnalogSensor(Sensor):
    def read(self):
        self._last = 512 / 1023 * 3.3
        return self._last

def print_value(sensor: Sensor):
    print("value:", sensor.read())

print_value(Sensor("temp"))
print_value(AnalogSensor("voltage", unit="V"))
```

زمان‌بندی و شبیه‌سازی تایمرها (ذهنیت میکروپایتون)

```
import time

def blink_simulation(on_time=0.5, off_time=0.5, cycles=5):
    for _ in range(cycles):
        print("LED ON")
        time.sleep(on_time)
        print("LED OFF")
        time.sleep(off_time)
```

```
blink_simulation()
```

کار با زمان، datetime، و فرمت لاگ

```
from datetime import datetime

def log_event(event, detail=""):
    ts = datetime.now().isoformat(timespec="seconds")
    print(f"[{ts}] {event} {detail}")

log_event("BOOT")
log_event("TEMP", "24.7C")
```

منابع کاربردی برای مطالعه بیشتر :

[آموزش رایگان پایتون سایت کدیاد](#)

[آموزش پایتون سایت w3schools](#)